MACHINE LEARNING / FOUNDATIONS

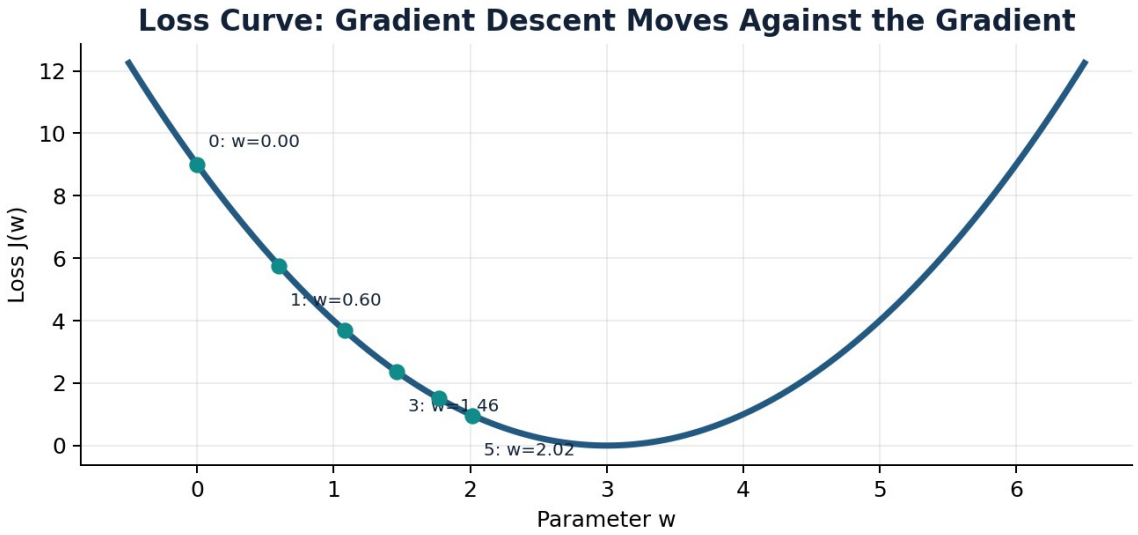
# Gradient Descent

## 梯度下降法：由零開始全面理解

從山谷直覺、數學公式與手算例子，到樓價預測、Python 實作及自我測驗。

**核心概念**

新參數 = 舊參數 − 學習率 × 梯度




*梯度下降會利用局部梯度資訊，逐步尋找較低的損失值。*

- 適合初學者的繁體中文講義
- 包含公式解說、比較表、圖表、Python 程式及附答案練習
- 可配合互動式 HTML 教材使用

**學習目標：理解「模型如何從錯誤中逐步調整參數」，而不只是背誦公式。**

# 本講義內容

建議依次閱讀；完成「互動實驗室」概念後，再用 Python 實作並完成測驗。

| 章節 | 主題 | 涵蓋內容 |
|---|---|---|
| 01 | 全貌：梯度下降在做甚麼 | 學習流程與生活比喻 |
| 02 | 五個核心概念 | 參數、損失、梯度、學習率、迭代 |
| 03 | 核心公式拆解 | 更新公式、導數正負號 |
| 04 | 逐步手算範例 | $J(w) = (w - 3)^2$ 的完整更新 |
| 05 | 互動實驗的數學理解 | 學習率如何影響收斂 |
| 06 | 應用：住宅樓價預測 | 多特徵線性回歸及梯度 |
| 07 | 三種梯度下降方法 | Batch、SGD、Mini-batch |
| 08 | 常見陷阱與停止條件 | 局部最低點、鞍點、過度擬合 |
| 09 | Momentum、RMSProp、Adam | 最佳化器與反向傳播 |
| 10 | Python 實作 | 可直接在 Google Colab 執行 |
| 11 | 自我測驗及答案 | 六題選擇題與解析 |
| 12 | 術語速查及延伸閱讀 | 中英對照詞彙表與官方資源 |

**閱讀提示**

文中的樓價與簡化數學例子用於解釋方法，不代表真實香港樓市的價格模型或投資建議。

# 01 全貌：梯度下降在做甚麼？

梯度下降法（Gradient Descent）是一種最佳化方法。在機器學習中，模型會先作出預測，再用損失函數量度預測與真實答案之間的差距；梯度下降利用梯度資訊，反覆更新模型參數，嘗試降低損失。

## 1.1 生活比喻：在濃霧中尋找山谷最低點

想像你站在一座被濃霧籠罩的山上，看不到整座山，也不知道最低點確切在哪裏。你只能觀察腳下附近的坡度，判斷向哪個方向會下坡，再走一小步，重新觀察，反覆進行。

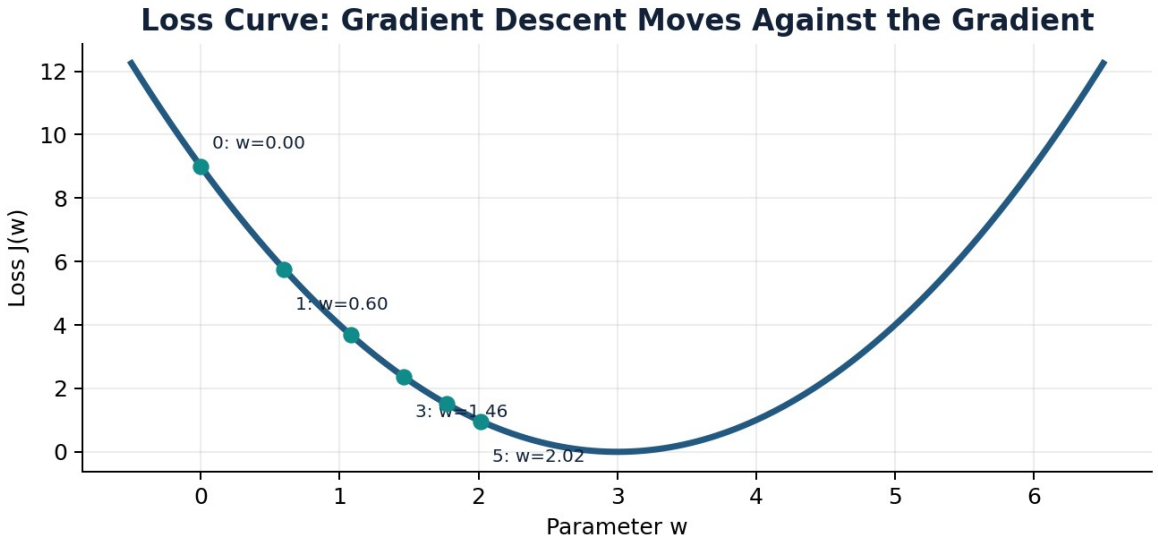



*圖 1. 損失曲線示意：每次更新後，參數位置可能逐步靠近較低點。*

| 山谷中的概念 | 機器學習概念 | 解釋 |
|---|---|---|
| 目前位置 | 模型參數（Parameters） | 例如線性模型的權重 w 和偏置 b。 |
| 山的高度 | 損失（Loss） | 反映模型的預測錯誤程度。 |
| 坡度與上坡方向 | 梯度（Gradient） | 描述損失在當前位置對參數的變化率。 |
| 每一步走多遠 | 學習率（Learning Rate） | 控制每次參數更新的步長。 |
| 反覆向下走 | 迭代更新（Iteration） | 重新計算梯度，繼續更新參數。 |
| 山谷較低處 | 低損失解／收斂 | 損失可能較低或更新逐漸穩定；不一定是全域最佳解。 |

## 1.2 在機器學習中的典型流程

1. 輸入訓練資料，例如住宅面積、地區和真實售價。
2. 模型用目前的參數產生預測。
3. 損失函數比較預測值與真實值，計算損失。
4. 計算損失對各參數的梯度。

5. 用梯度與學習率更新參數，再重複以上步驟。

**最重要的理解**

梯度下降不是令 AI 一次猜中答案，而是利用計算與反覆更新，逐步尋找較好的模型參數。

# 02 五個核心概念

## 2.1 參數 Parameters：模型可以調整的值

$$\hat{y} = wx + b$$

ŷ：模型預測；x：輸入特徵；w：權重；b：偏置。

例如用實用面積 x 預測樓價，w 控制面積變化對預測的影響，b 則調整預測的基準值。模型訓練的其中一項工作，就是尋找較合適的 w 和 b。

## 2.2 損失函數 Loss Function：模型錯得有多嚴重？

損失函數把模型的預測與目標答案轉換成一個數值。數值愈低，通常代表模型在指定損失標準下表現愈好。不同任務可以使用不同損失函數。

$$J(w,b) = (1/n)\ \Sigma_i\ (\hat{y}_i - y_i)^2$$

均方誤差（MSE），常用於回歸；n 為樣本數。

| 真實值（萬元） | 預測值（萬元） | 誤差（預測 − 真實） | 平方誤差（萬元²） |
|---|---|---|---|
| 500 | 400 | −100 | 10,000 |
| 500 | 450 | −50 | 2,500 |
| 500 | 500 | 0 | 0 |
| 500 | 550 | +50 | 2,500 |
| 500 | 600 | +100 | 10,000 |

表中的平方誤差使用「萬元」作為數值單位，所以平方後的單位是「萬元²」，不是直接的貨幣金額。MSE 會計算所有樣本平方誤差的平均值。分類模型常用交叉熵（Cross-Entropy），不一定使用 MSE。

## 2.3 導數與梯度 Derivative and Gradient

導數表示單一變數函數在某個位置的瞬時變化率。多參數函數會對每個參數計算偏導數，再把它們組成梯度向量。梯度指向局部上升最快的方向；若要降低損失，就沿着反方向更新。

$$dJ/dw$$

單一參數 w 的導數，代表損失對 w 的局部變化率。

$$\nabla J(w,b) = [\ \partial J/\partial w\ ,\ \partial J/\partial b\ ]^T$$

多個參數的梯度向量。

| 當前導數 | 局部解讀 | 基本更新方向（α > 0） |
| --- | --- | --- |
| 正數 | 增加 w 會在局部令損失上升 | 減少 w |
| 負數 | 增加 w 會在局部令損失下降 | 增加 w |
| 接近零 | 局部斜度接近水平 | 更新量通常較小 |

## 2.4 學習率 Learning Rate：每一步走多遠？

學習率通常以 α 表示，是控制參數更新幅度的超參數。它不是模型從資料直接學到的普通權重，而是訓練時設定或由排程調整的值。

| 設定 | 比喻 | 常見影響 |
| --- | --- | --- |
| 太小 | 每次只走極小一步 | 學習很慢，需要更多次迭代。 |
| 合適 | 每次走適當距離 | 在相應條件下較穩定地接近低點。 |
| 太大 | 大步跨過低點 | 可能震盪、令損失增加或發散。 |

## 2.5 迭代、批次與 Epoch

| 術語 | 意思 |
| --- | --- |
| Iteration（迭代） | 一次參數更新步驟。 |
| Batch（批次） | 用來計算某次梯度的一組訓練樣本。 |
| Epoch（訓練週期） | 訓練集完整被模型使用一次。 |
| Convergence（收斂） | 參數或損失逐漸穩定；不保證已找到全域最低點。 |

# 03 核心公式拆解

$$\theta_{t+1} = \theta_t - \alpha \nabla J(\theta_t)$$

新參數 = 舊參數 − 學習率 × 當前梯度。

| 符號 | 意思 | 直覺理解 |
| --- | --- | --- |
| $\theta_t$ | 目前模型參數 | 目前所站的位置。 |
| $J(\theta)$ | 損失函數 | 地勢高度。 |
| $\nabla J(\theta_t)$ | 目前梯度 | 局部上坡最快的方向。 |
| $\alpha$ | 學習率 | 每一步的長度。 |
| 負號 − | 沿梯度的反方向更新 | 嘗試走向局部下坡。 |
| $t+1$ | 下一次更新 | 下一個參數版本。 |

## 3.1 導數為正與為負時會發生甚麼？

假設學習率 $\alpha$ 是正數：

$$\text{若梯度} > 0：\ w_{new} = w_{old} - \alpha \times \text{正數}$$

更新通常減少 w。

$$\text{若梯度} < 0：\ w_{new} = w_{old} - \alpha \times \text{負數}$$

減去負數等於加上正數，更新通常增加 w。

**小心這個細節**

梯度的符號幫助判斷局部方向，但步長仍然重要。若學習率太大，即使方向一開始合理，更新也可能跨過低點而令損失反彈。

# 04 逐步手算：讓權重從 0 接近 3

這個最小例子用來看清梯度下降如何運作。只有一筆資料：輸入 $x = 1$、真實答案 $y = 3$。模型為 $\hat{y} = wx$，初始權重 $w = 0$，學習率 $\alpha = 0.1$。

## 第一步：定義損失函數

$$J(w) = (w - 3)^2$$

因為 $x = 1$、$\hat{y} = w$，真實目標 $y = 3$。

當 $w = 0$ 時，預測為 0，平方誤差是 $(0 - 3)^2 = 9$。目標是找到令損失 $J(w)$ 下降的權重。

## 第二步：求導

$$dJ/dw = 2(w - 3)$$

把 $w = 0$ 代入，梯度為 $2(0 - 3) = -6$。梯度為負，所以更新時「減去負數」會令權重增加。

## 第三步：更新權重

$$w_{new} = 0 - 0.1 \times (-6) = 0.6$$

新損失為 $(0.6 - 3)^2 = 5.76$，比原本的 9 小。再用新的 $w$ 重新計算梯度，就能進行下一輪更新。

## 第四步：反覆更新

| 迭代 | 舊權重 w | 梯度 2(w − 3) | 新權重 | 新損失 |
|---|---|---|---|---|
| 0 | 0.00000 | -6.00000 | 0.60000 | 5.76000 |
| 1 | 0.60000 | -4.80000 | 1.08000 | 3.68640 |
| 2 | 1.08000 | -3.84000 | 1.46400 | 2.35930 |
| 3 | 1.46400 | -3.07200 | 1.77120 | 1.50995 |
| 4 | 1.77120 | -2.45760 | 2.01696 | 0.96637 |
| 5 | 2.01696 | -1.96608 | 2.21357 | 0.61848 |

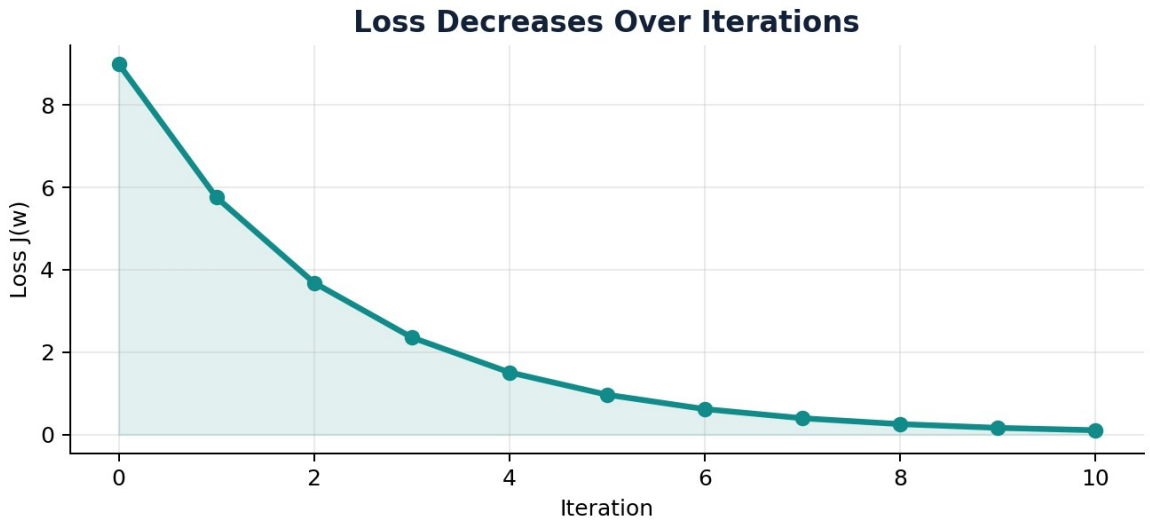



*圖 2. 在此例中，損失由 9 開始，隨更新逐步下降。*

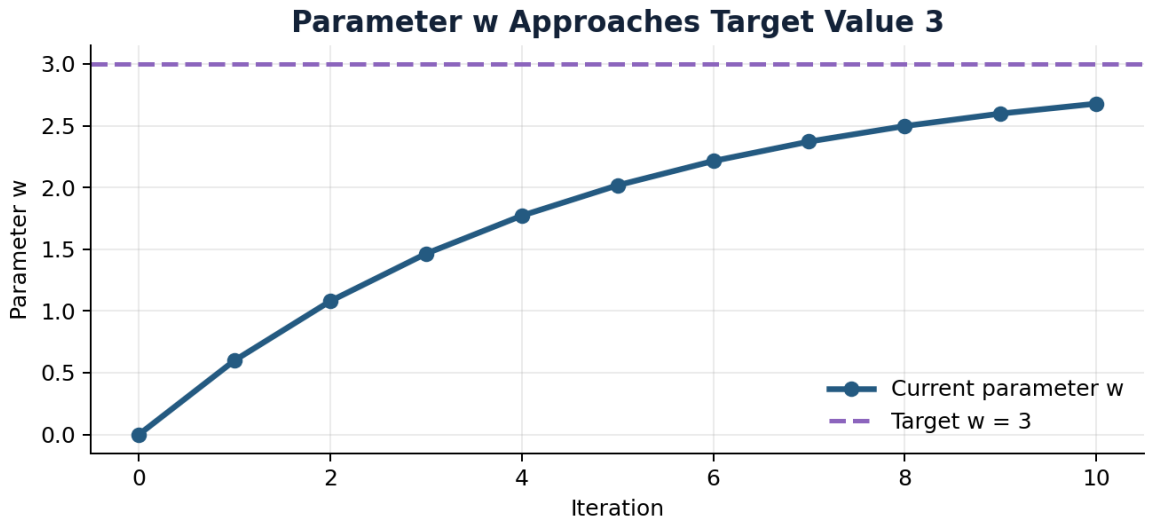



*圖 3. 權重逐步接近目標值 3；有限次更新後仍可能未完全相等。*

**這個例子要記住的事**

當參數逐漸接近最低點，梯度的絕對值也會減小，因此更新幅度一般會逐漸變小。真實訓練不一定要求損失恰好等於零。

# 05 互動實驗的數學理解：學習率如何影響收斂？

互動 HTML 教材可調整學習率、初始權重及更新次數。本章把實驗背後的規律寫清楚，方便離線閱讀與複習。

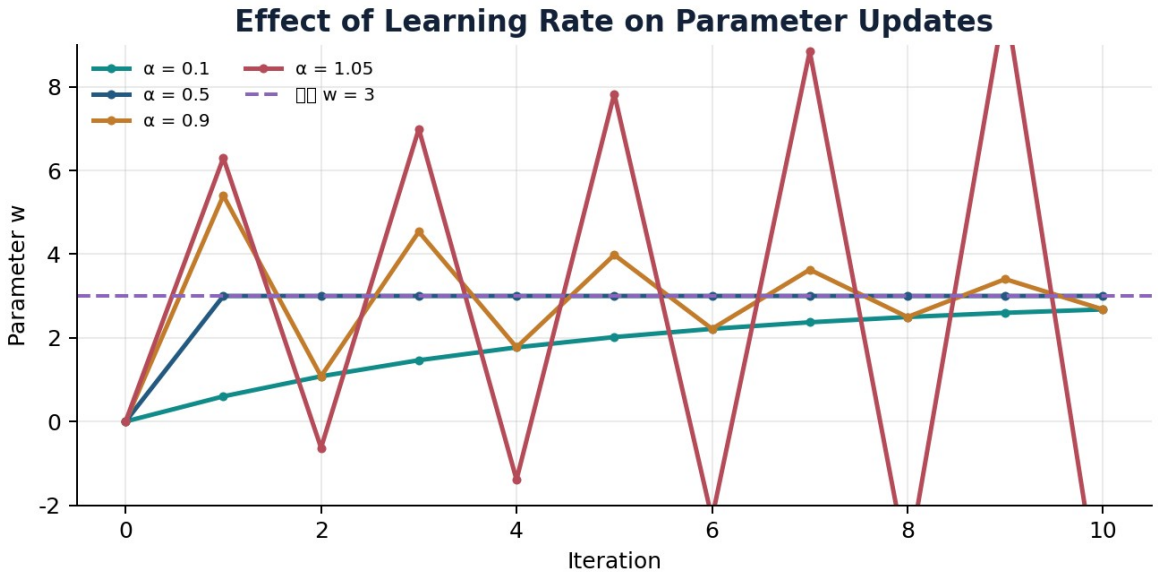



*圖 4. 相同函數下，不同學習率可能造成不同的收斂速度或震盪情況。*

對 $J(w) = (w - 3)^2$，梯度是 $2(w - 3)$，更新式可以改寫成：

$$w_{new} - 3 = (1 - 2\alpha)(w_{old} - 3)$$

所以，誤差每次會乘上 $(1 - 2\alpha)$。在這個特定的一維二次函數中，若 $0 < \alpha < 1$，誤差會逐步縮小；$\alpha = 0.5$ 時，一次便到達 $w = 3$；$\alpha = 1$ 時在 0 與 6 之間來回震盪；$\alpha > 1$ 時誤差幅度會擴大。這是本例的數學結果，不可直接套用到所有模型。

| 學習率 α | 本例中的典型現象 | 說明 |
|---|---|---|
| 0.01 | 穩定但很慢 | 每次修正很小。 |
| 0.10 | 逐步靠近最低點 | 更新幅度適中。 |
| 0.50 | 一步到達 w = 3 | 只對這個特定二次函數成立。 |
| 0.90 | 跨越目標但振幅逐漸減小 | 仍會逐步收斂。 |
| 1.00 | 在目標兩側震盪 | 本例不會收斂到 3。 |
| 大於 1 | 誤差幅度擴大 | 本例會發散。 |

### 實務建議

先設定合理的起始學習率，觀察訓練損失及驗證表現，再調整。Learning-rate scheduling 可以按訓練進度降低學習率；warm-up 則在訓練初期先逐步增加學習率。

# 06 實際應用：住宅樓價預測

現實模型往往用多個特徵預測目標，例如用住宅面積、距離港鐵站的距離及其他住宅特徵估算售價。以下只用作數學教學示範，真實樓價預測還會受地區、樓齡、樓層、景觀、市場週期、成交條件等因素影響。

$$\hat{y} = w_1x_1 + w_2x_2 + b$$

$x_1$：實用面積；$x_2$：距離港鐵站；$\hat{y}$：預測樓價。

| 變數 | 角色 | 如何理解 |
| --- | --- | --- |
| $x_1$ | 輸入特徵 | 例如實用面積。 |
| $x_2$ | 輸入特徵 | 例如距離港鐵站的距離。 |
| $w_1$、$w_2$ | 權重 | 模型學到各特徵與預測值之間的關係。 |
| $b$ | 偏置 | 調整預測的基準值。 |
| $\hat{y}$ | 預測輸出 | 模型估算的樓價，而非保證的實際成交價。 |

## 6.1 計算多個參數的梯度

$$J(w_1,w_2,b) = (1/n)\ \Sigma_i\ (\hat{y}_i - y_i)^2$$

$$\partial J/\partial w_1 = (2/n)\ \Sigma_i\ (\hat{y}_i - y_i)x_{1i}$$

$$\partial J/\partial w_2 = (2/n)\ \Sigma_i\ (\hat{y}_i - y_i)x_{2i}$$

$$\partial J/\partial b = (2/n)\ \Sigma_i\ (\hat{y}_i - y_i)$$

模型會分別以這些偏導數更新 $w_1$、$w_2$ 和 $b$。每個梯度反映對應參數在目前位置對損失的局部影響。若特徵量度尺度差異很大，例如平方呎、米、百萬元，適當的特徵標準化通常有助改善最佳化的數值條件。

## 6.2 重要限制

- 梯度下降只會根據既定模型、損失函數及訓練資料更新參數。
- 若資料有偏差、資料不足或缺失重要特徵，模型仍可能預測失準。
- 訓練損失低不代表新資料一定準確；需要利用驗證集或測試集評估泛化能力。

# 07 三種主要梯度下降方法

這三種方法的主要差別，是每次更新前使用多少訓練樣本來估算梯度。

| 方法 | 每次更新使用的樣本 | 優點 | 限制 |
| --- | --- | --- | --- |
| Batch Gradient Descent（批次） | 整個訓練集 | 梯度通常較穩定。 | 資料量很大時，一次更新成本可能很高。 |
| Stochastic Gradient Descent（SGD，隨機） | 典型定義下為單一樣本 | 單次更新計算量小。 | 梯度波動較大，損失可能上落。 |
| Mini-batch Gradient Descent（小批次） | 一小批樣本，例如 32、64、128 | 常能兼顧計算效率及梯度穩定性。 | 需選擇合適的 batch size。 |

## 7.1 例子：有 10,000 筆訓練資料

| 方法 | 每次更新的樣本數 | 一個 Epoch 的典型更新次數 |
| --- | --- | --- |
| Batch | 10,000 | 1 |
| SGD | 1 | 10,000 |
| Mini-batch，batch size = 100 | 100 | 100 |

以上假設每個 Epoch 完整處理一次資料，並沒有丟棄最後不足一批的樣本。實際更新次數會受資料切批方式、梯度累積及軟件設定影響。SGD 一詞有時會被廣義用來指隨機最佳化家族；在深度學習軟件中，SGD 亦是具體最佳化器名稱。

# 08 常見陷阱與停止條件

## 8.1 常見問題

| 問題 | 為甚麼會出現 | 處理方向 |
| --- | --- | --- |
| 學習率過小 | 每次更新幅度有限，進展緩慢。 | 嘗試不同學習率或使用排程。 |
| 學習率過大 | 更新可能越過低點，震盪或發散。 | 降低學習率，檢查梯度尺度及損失曲線。 |
| 特徵尺度差異大 | 參數方向上的尺度不同，可能令最佳化困難。 | 按資料情況作標準化或縮放特徵。 |
| 局部最低點 | 某點比附近點低，但不是全域最低點。 | 可檢查初始化、最佳化器及多次訓練結果。 |
| 鞍點 | 某些方向上升，另一些方向下降；梯度可能為零。 | 使用適當最佳化策略，並理解局部梯度限制。 |
| 平坦區域 | 局部梯度很小，更新可能緩慢。 | 檢查模型、初始化、學習率與訓練曲線。 |
| 非凸損失 | 可能存在多個局部低點、鞍點與平坦區域。 | 不要假定一定找到全域最低點。 |
| 過度擬合 | 模型過度貼合訓練資料，新資料表現反而不佳。 | 觀察驗證集、正則化或使用 Early Stopping。 |

## 8.2 何時停止訓練？

- 梯度大小低於預先設定的門檻。
- 損失在多次迭代中幾乎沒有改善。
- 達到預設的最大迭代次數或訓練預算。
- 驗證集表現不再改善，觸發早停（Early Stopping）。

> **注意**
>
> 損失不再下降，可能是已接近低點，也可能因學習率太小或遇到平坦區域。停止條件應配合驗證表現和整體訓練情況判斷。

## 8.3 凸函數與非凸函數

對可微凸函數，在合適條件及適當設定下，梯度下降可以收斂至全域最低點。一般神經網絡常涉及非凸損失函數，不能保證每次都找到全域最佳解。這是理論性質與實務訓練之間的重要區別。

# 09 Momentum、RMSProp、Adam 與反向傳播

| 方法／概念 | 核心想法 | 常見作用或分工 |
| --- | --- | --- |
| Momentum（動量法） | 將過去梯度資訊累積為動量。 | 在一些情況下減少震盪，並加快一致方向的移動。 |
| RMSProp | 用梯度平方的移動平均調整有效步長。 | 讓不同參數的有效更新尺度可以不同。 |
| Adam | 結合梯度一階與二階動量估計，並作偏差修正。 | 常用於深度學習；但不保證所有任務表現最佳。 |
| Backpropagation（反向傳播） | 用鏈式法則高效率計算神經網絡各參數的梯度。 | 負責計算梯度，不等同於參數更新本身。 |
| Optimizer（最佳化器） | 依梯度等資訊更新模型參數。 | 可採用 SGD、Momentum、RMSProp、Adam 等規則。 |

> **分清三個角色**
>
> 損失函數負責評估預測好壞；反向傳播負責計算梯度；梯度下降或其他最佳化器則利用梯度更新參數。

## 9.1 梯度下降配合哪些模型？

| 模型 | 主要可訓練參數 | 常見損失函數 |
| --- | --- | --- |
| 線性回歸（Linear Regression） | 權重 w、偏置 b | 均方誤差 MSE 等。 |
| 邏輯回歸（Logistic Regression） | 分類權重與偏置 | 交叉熵等。 |

| 模型 | 主要可訓練參數 | 常見損失函數 |
| --- | --- | --- |
| 神經網絡（Neural Networks） | 多層網絡權重與偏置 | 視任務而定，例如交叉熵或 MSE。 |
| 深度學習模型 | 大量可訓練參數 | 按分類、回歸、生成等任務選擇。 |

# 10 Python 實作：從零執行梯度下降

以下範例只使用 Python 內置功能，不需要安裝任何額外套件。你可以將程式複製到 Google Colab、Jupyter Notebook 或其他 Python 環境。這是單參數示範；真實訓練通常涉及多筆樣本、多個參數、驗證資料及更複雜的損失函數。

```
# Gradient Descent 梯度下降法
# 目標：找出令 J(w) = (w - 3) ** 2 最小的 w
w = 0.0
learning_rate = 0.1
max_iterations = 100
for step in range(max_iterations):
    # 1. 計算目前的損失
    loss = (w - 3) ** 2
    # 2. 計算目前的梯度
    gradient = 2 * (w - 3)
    # 3. 每隔 10 次顯示一次結果
    if step % 10 == 0:
        print(
            f"Step: {step:3d} | "
            f"w: {w:.6f} | "
            f"Loss: {loss:.6f} | "
            f"Gradient: {gradient:.6f}"
        )
    # 4. 梯度非常小時停止
    if abs(gradient) < 1e-6:
        print("梯度已接近零，停止更新。")
        break
    # 5. 更新權重
    w = w - learning_rate * gradient
print(f"\n最終權重 w = {w:.6f}")
print(f"最終損失 J(w) = {(w - 3) ** 2:.8f}")
```

## 10.1 逐行理解

**`w = 0.0`** — 設定初始權重。

**`loss = (w - 3) ** 2`** — 計算目前的平方誤差。

**`gradient = 2 * (w - 3)`** — 計算損失對權重的導數。

**`abs(gradient) < 1e-6`** — 檢查梯度是否足夠小，符合此範例的停止條件。

**`w = w - learning_rate * gradient`** — 按梯度下降規則更新權重。

# 11 自我測驗：六題檢查理解

## 1. 梯度的方向代表甚麼？

◯ A. 損失下降最快的方向

◯ B. 損失上升最快的方向

◯ C. 模型預測的答案

**答案：B。** 梯度指向損失上升最快的局部方向，梯度下降沿相反方向更新。

## 2. 更新公式為甚麼有負號？

◯ A. 令參數永遠為負數

◯ B. 沿梯度反方向更新

◯ C. 令損失永遠等於零

**答案：B。** 負號令更新朝向局部損失下降的反梯度方向。

## 3. 學習率太大可能造成甚麼問題？

◯ A. 永遠不會更新參數

◯ B. 一定找到全域最低點

◯ C. 越過最低點、震盪或發散

**答案：C。** 步長過大可能使更新不穩定，甚至令損失持續增加。

## 4. Loss Function 的主要用途是甚麼？

◯ A. 量度模型預測的錯誤

◯ B. 決定資料有多少列

◯ C. 自動增加訓練資料

**答案：A。** 損失函數比較模型預測與目標答案，提供最佳化依據。

## 5. 每次用 100 筆資料更新參數，屬於哪種方法？

◯ A. Batch（使用全部資料）

◯ B. Mini-batch（小批次）

◯ C. 必定是單樣本 SGD

**答案：B。** 若訓練集大於 100 筆，每次取 100 筆就是小批次梯度下降。

## 6. 訓練損失降低，是否代表新資料預測一定更準？

◯ A. 是，一定更準

◯ B. 不是，需要檢查驗證集

◯ C. 只要梯度為正就會更準

**答案：B。** 模型可能過度擬合訓練資料，需要檢查未參與訓練的驗證資料。

# 12 術語速查表與延伸閱讀

| English | 繁體中文 | 簡明解釋 |
|---|---|---|
| Gradient Descent | 梯度下降法 | 沿梯度反方向逐步更新參數的最佳化方法。 |
| Gradient | 梯度 | 多參數函數的偏導數向量，指向局部上升最快方向。 |
| Derivative | 導數 | 單一變數函數在某位置的瞬時變化率。 |
| Loss Function | 損失函數 | 量度預測與目標答案之間差距的函數。 |
| Parameter | 參數 | 模型透過訓練調整的值，例如權重與偏置。 |
| Learning Rate | 學習率 | 控制每次參數更新幅度的超參數。 |
| Iteration | 迭代 | 一次參數更新步驟。 |
| Epoch | 訓練週期 | 訓練集完整被使用一次。 |
| Batch Size | 批次大小 | 一次估算梯度使用的樣本數量。 |
| Convergence | 收斂 | 損失或參數逐漸穩定。 |
| Local Minimum | 局部最低點 | 比附近點低，但不一定是全域最低點。 |
| Saddle Point | 鞍點 | 某些方向上升、另一些方向下降的臨界點。 |
| Backpropagation | 反向傳播 | 用鏈式法則計算神經網絡梯度的方法。 |
| Overfitting | 過度擬合 | 模型過度貼合訓練資料，對新資料泛化不佳。 |
| Optimizer | 最佳化器 | 利用梯度等資訊更新模型參數的方法或演算法。 |
| Adam | Adam 最佳化器 | 結合一階與二階動量估計的常用梯度型最佳化器。 |

## 建議學習路線

1. 先用「山谷直覺」建立方向感。
2. 記熟更新公式，理解導數正負號與學習率。
3. 手算 $J(w) = (w - 3)^2$ 的前幾次更新。
4. 使用互動式 HTML 教材嘗試不同學習率。
5. 在 Google Colab 執行 Python，改變 learning_rate 和 max_iterations。
6. 完成測驗，再進一步學習線性回歸、邏輯回歸與神經網絡。

## 延伸閱讀

Google Machine Learning Crash Course：Gradient Descent（官方英文教材）

https://developers.google.com/machine-learning/crash-course/linear-regression/gradient-descent

**最後總結：** 目標是減少損失；梯度指出局部上坡最快方向；學習率控制步長；梯度下降沿反方向更新參數，並以訓練與驗證表現判斷何時停止。

# 附錄：配合互動式 HTML 教材使用

本講義可配合另一個單一 HTML 檔案「gradient_descent_interactive_zh.html」使用。該網頁包含多個分頁：概覽、山谷直覺、核心概念、公式拆解、互動實驗室、逐步計算、三種方法、陷阱與最佳化器、Python 實作、小測驗及詞彙表。

| 互動頁面 | 建議做法 |
| --- | --- |
| 互動實驗室 | 調整學習率、初始權重與更新次數，對照本講義第 05 章。 |
| 逐步計算 | 比對每輪的舊權重、梯度、新權重與損失。 |
| Python 實作 | 複製程式碼至 Google Colab 執行並修改參數。 |
| 小測驗 | 先獨立作答，再閱讀本講義第 11 章答案解析。 |

注意：HTML 互動教材的視覺化是為簡化教學而設計；真實模型的最佳化情況會因損失函數、資料規模、參數維度及最佳化器而不同。